

在 AI 時代建構應用程式

來源：<https://codelabs.developers.google.com/codelabs/building-applications-in-the-ai-era?hl=zh-tw>

步驟數：15

目錄

1. 總覽
2. 必要條件
3. 專案設定
4. 啟用及使用 Gemini Cloud Assist
5. 在 Vertex AI Agent Builder 中為烹飪建議聊天機器人建立搜尋應用程式
6. 在 Colab Enterprise 筆記本中載入及清理資料，並使用 Gemini Code Assist 輔助
7. 在 Vertex AI Agent Builder 中建立食譜生成器的搜尋應用程式
8. 開啟 Cloud Shell 編輯器
9. 啟用 API
10. 在 Gemini 的協助下，規劃 Python 和 Streamlit 網頁應用程式的核心架構
11. 將網頁應用程式部署至 Cloud Run
12. 將「烹飪建議」頁面連結至我們的食譜搜尋 Agent Builder 應用程式
13. (選用) 將食譜搜尋頁面連結至食譜搜尋 Agent Builder 應用程式
14. (選用) 探索最終應用程式
15. 恭喜！

1. 總覽

在本實驗室中，您將使用 Google 的生成式 AI 產品，在 Gemini Cloud Assist 的輔助下，於 Google Cloud 中建構基礎架構；使用 Data Canvas 的自然語言轉 SQL 功能查詢 BigQuery 資料；在 Gemini Code Assist 的輔助下，於 Colab Enterprise Jupyter 筆記本和 Eclipse Theia (Visual Studio Code) 中編寫程式碼；以及在 Vertex AI Agent Builder 中，整合以 Cloud Storage 和 BigQuery 基礎來源為基礎的 AI 搜尋和即時通訊功能。

我們的目標是建立名為「AI Recipe Haven」的食譜和烹飪網站。這個網站會以 Python 和 Streamlit 建構，並包含兩個主要頁面。「烹飪建議」會代管我們使用 Gemini 和 Vertex AI Agent Builder 建立的聊天機器人，並以一組食譜做為基礎來源，提供烹飪建議及回答烹飪相關問題。食譜搜尋功能是由 Gemini 驅動的搜尋引擎，這次的基礎是 BigQuery 食譜資料庫。

如果您在練習中遇到任何程式碼問題，所有程式碼檔案的解決方案都位於練習的 GitHub 存放區中，也就是 [solution](#) 分支。

目標

在本實驗室中，您將瞭解如何執行下列工作：

- 啟用及使用 Gemini Cloud Assist
 - 在 Vertex AI Agent Builder 中為烹飪建議聊天機器人建立搜尋應用程式
 - 在 Colab Enterprise 筆記本中載入及清理資料，並使用 Gemini Code Assist 輔助
 - 在 Vertex AI Agent Builder 中建立食譜生成器的搜尋應用程式
 - 在 Gemini 的協助下，規劃 Python 和 Streamlit 網頁應用程式的核心架構
 - 將網頁應用程式部署至 Cloud Run
 - 將「烹飪建議」頁面連結至我們的食譜搜尋 Agent Builder 應用程式
 - (選用) 將食譜搜尋頁面連結至食譜搜尋 Agent Builder 應用程式
 - (選用) 探索最終應用程式
-

步驟 2 · 約 0 分鐘

2. 必要條件

1. 如果沒有 Google 帳戶，請先[建立帳戶](#)。
 - 請改用個人帳戶，而非公司或學校帳戶。公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。
-

3. 專案設定

1. 登入 [Google Cloud 控制台](#)。
 2. 在 Cloud 控制台中[啟用帳單](#)。
 - 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
 - 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
 - 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。
 - 參加虛擬實作實驗室活動？您可能可以獲得 [\\$5 美元抵免額](#)。
 3. [建立新專案](#)，或選擇重複使用現有專案。
 4. 確認 Cloud Billing 的「我的專案」
 - 如果新專案的「Billing account」欄顯示 `Billing is disabled`，請按照下列步驟操作：
 1. 按一下「Actions」欄中的三點圖示
 2. 按一下「變更帳單」
 3. 選取要使用的帳單帳戶
 - 如果您參加的是現場活動，帳戶名稱可能為「Google Cloud Platform 試用帳單帳戶」
-

4. 啟用及使用 Gemini Cloud Assist

在這項工作中，我們將啟用並使用 Gemini Cloud Assist。在 Google Cloud 控制台中工作時，Gemini Cloud Assist 可以提供建議、協助您建構、設定及監控 Google Cloud 基礎架構，甚至建議 `gcloud` 指令及編寫 Terraform 指令碼。

1. 如要啟用 Cloud Assist，請點選 Cloud 控制台 UI 頂端的「搜尋」方塊，然後選取「詢問 Gemini」或「詢問 Gemini for Cloud 控制台」。
2. 捲動至頁面的「必要 API」部分，然後**啟用** Gemini for Google Cloud API。
3. 如果沒有立即看到即時通訊介面，請按一下「開始即時通訊」。首先，請 Gemini 說明使用 Cloud Shell 編輯器有哪些好處。請花幾分鐘時間查看生成的回覆。
4. 接著，請詢問 Agent Builder 的優點，以及如何協助生成回覆。
5. 最後，我們來看看比較結果。在 Google Cloud 控制台的 Gemini 對話視窗中，提出下列問題：

What are the major steps to creating a search app grounded in a GCS data source using Vertex AI Agent builder?

5. 在 Vertex AI Agent Builder 中為烹飪建議聊天機器人建立搜尋應用程式

我們即將建構的網站會提供烹飪建議頁面，其中包含聊天機器人，可協助使用者尋找烹飪相關問題的解答。這項功能會以 Gemini 為基礎，並參考 70 本公版食譜。Gemini 回答問題時，會以食譜為準。

1. 使用 Cloud 控制台搜尋方塊前往 **Vertex AI**。在資訊主頁中，點按「啟用所有建議的 API」。請稍候片刻。如果系統彈出視窗，指出需要啟用 Vertex AI API，請一併**啟用**。啟用 API 後，即可前往下一個步驟。
2. 使用搜尋功能前往 **Agent Builder**，然後點選「繼續並啟用這個 API」。
3. 如 Gemini 在先前的建議中指出，在 Agent Builder 中建立搜尋應用程式的第一步，是建立權威資料來源。使用者搜尋時，Gemini 會瞭解問題和智慧回覆的撰寫方式，但會從基準來源尋找回覆中使用的資訊，而不是從內建知識庫提取。從左側選單前往「資料儲存庫」，然後選取「建立資料儲存庫」。
4. 我們目前使用的公共領域食譜位於外部專案的 Cloud Storage bucket 中，**選取** Cloud Storage 來源類型。
5. 檢查與要匯入資訊類型相關的預設選項，但不要變更。將匯入類型設為「資料夾」，並使用 `labs.roitraining.com/labs/old-cookbooks` 做為 bucket 路徑，然後點選「繼續」。
6. 將資料儲存庫命名為 `old-cookbooks`。按一下「EDIT」(編輯)，將 ID 變更為 `old-cookbooks-id`，然後「Create」(建立) 資料儲存庫。

Vertex AI Agent Builder 支援多種應用程式類型，而資料儲存庫是各類型的真實來源。搜尋應用程式適合一般用途和搜尋。Chat 應用程式適用於 Dataflow 驅動的聊天機器人/語音機器人應用程式中的生成流程。建議應用程式有助於建立更完善的建議引擎。而 Agent 應用程式則是用於建立生成式 AI 驅動的代理程式。最終，Agent 可能最能滿足我們的需求，但目前產品仍處於預覽階段，因此我們將繼續使用 Search 應用程式類型。

7. 使用左側選單前往「應用程式」，然後按一下「建立新應用程式」。
8. 按一下「網站搜尋功能」資訊卡上的「建立」。將應用程式命名為 `cookbook-search`。按一下「編輯」，然後將應用程式 ID 設為 `cookbook-search-id`。將公司設為 `Google`，然後按一下「繼續」。
9. 檢查您在幾個步驟前建立的 **old-cookbooks** 資料儲存庫，然後「建立」搜尋應用程式。

查看「活動」分頁時，您可能會發現系統仍在匯入食譜並建立索引。Agent Builder 需要 5 分鐘以上的時間，才能為我們提供的 70 本食譜中的數千個頁面建立索引。在等待期間，我們來載入並清除食譜資料庫資料，以供食譜產生器使用。

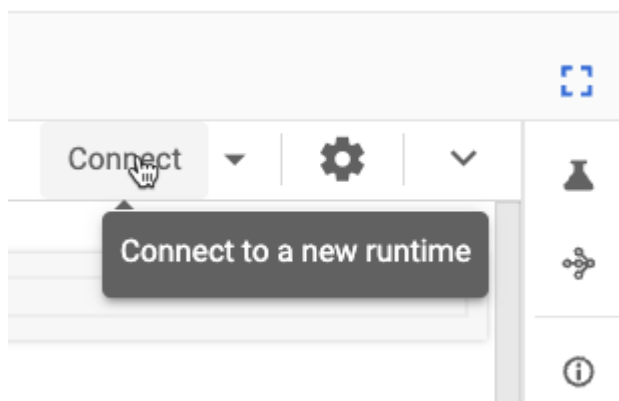
步驟 6 · 約 0 分鐘

6. 在 Colab Enterprise 筆記本中載入及清理資料，並使用 Gemini Code Assist 輔助

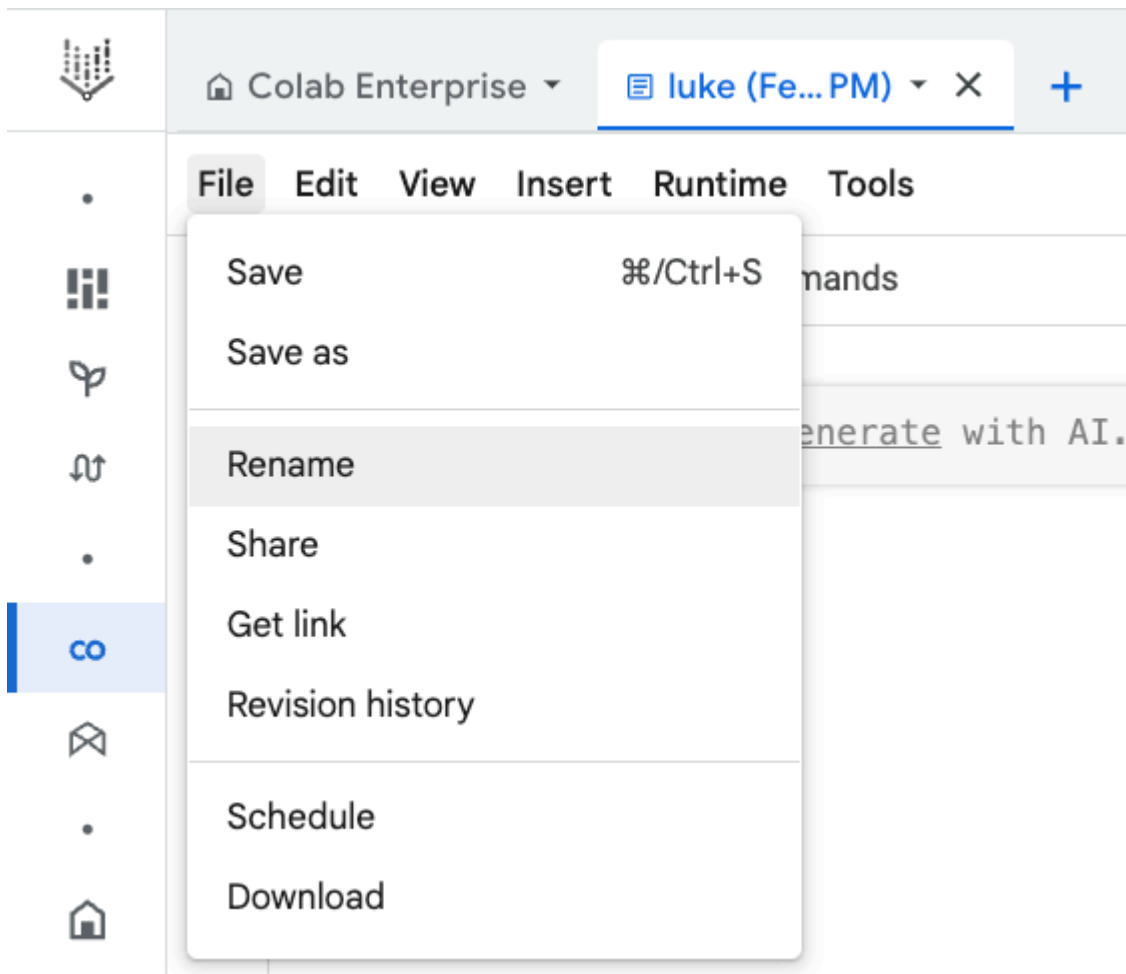
Google Cloud 提供幾種主要方式，讓您使用 Jupyter 筆記本。我們將使用 Google 的最新產品 Colab Enterprise。部分使用者可能熟悉 Google 的 [Colab](#) 產品，個人和機構通常會使用這項產品，在免費環境中試用 Jupyter 筆記本。Colab Enterprise 是 Google Cloud 的商業產品，與其他 Google 雲端產品完全整合，並充分運用 GCP 環境的安全防護和法規遵循功能。

Colab Enterprise 提供多項功能，其中一項是整合 Google 的 Gemini Code Assist。您可以在多種程式碼編輯器中使用 Code Assist，在編寫程式碼時取得建議和 [內嵌](#) 建議。我們會在處理食譜資料時運用這項生成式助理。

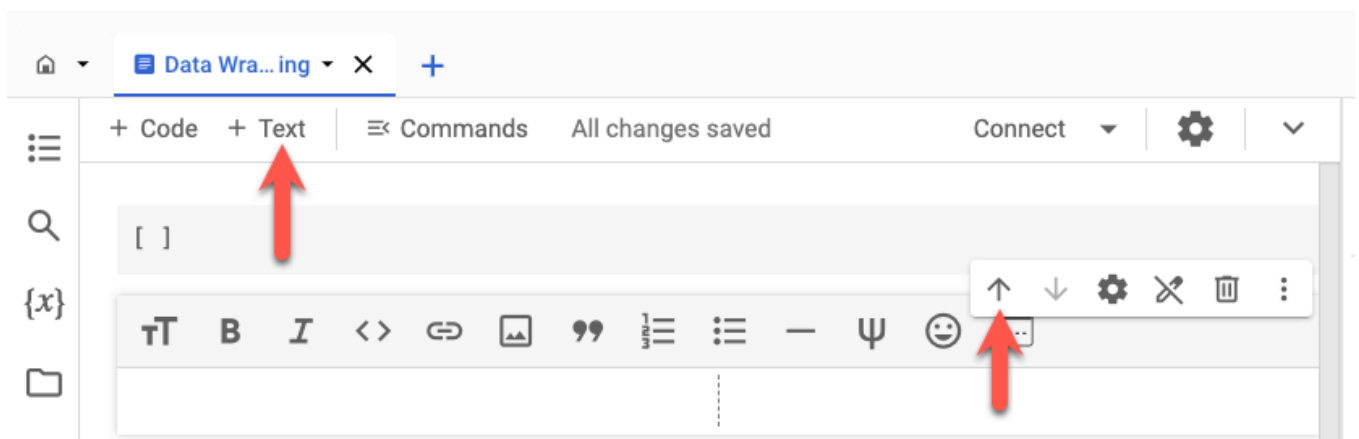
1. 使用搜尋功能前往 **Colab Enterprise**，然後按一下「建立筆記本」。如果系統顯示邀請你試用新版 Colab 功能的訊息，請關閉該訊息。如要啟動執行階段 (筆記本背後的運算能力)，請按一下新筆記本右上角的「連線」。



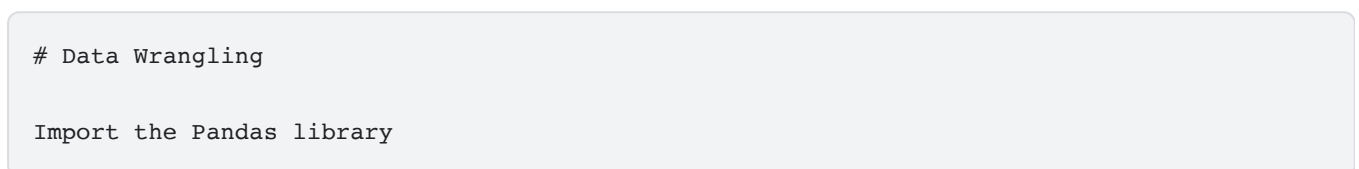
2. 依序點選「File」>「Rename」，將筆記本重新命名為 `Data Wrangling`。



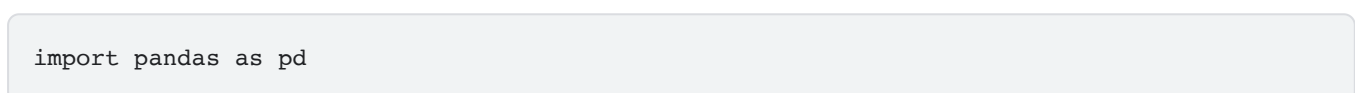
3. 按一下「+ 文字」建立新的文字方塊，然後使用向上箭頭將其移至頁面上的第一個儲存格。



4. 編輯文字方塊並輸入：



5. 在剛建立的文字區塊下方的程式碼區塊中，開始輸入 `imp`，Gemini Code Assist 應會以灰色建議其餘匯入內容。按下 **Tab** 鍵即可接受建議。



6. 在匯入程式碼方塊下方，建立另一個文字方塊並輸入：

```
Create a Pandas DataFrame from:  
gs://labs.roitraining.com/labs/recipes/recipe_dataset.csv. View the first few records.
```

7. 建立及編輯其他程式碼區塊。再次開始輸入 `df =`，並檢查 Gemini Code Assistant 生成的程式碼。如果系統在生成的建議上方顯示 Python 關鍵字的自動完成下拉式清單，請按下 **Esc** 鍵，查看淺灰色的建議程式碼。再次按下 **Tab** 鍵接受建議。如果建議未包含 `head()` 函式呼叫，請新增。

```
df = pd.read_csv('gs://labs.roitraining.com/labs/recipes/recipe_dataset.csv')  
df.head()
```

8. 按一下第一個程式碼儲存格 (您在其中匯入 Pandas)，然後使用「Commands」選單或鍵盤執行所選儲存格。在鍵盤上按下 **Shift + Enter** 鍵，即可執行儲存格並將焦點移至下一個儲存格 (視需要建立儲存格)。請等待儲存格執行完畢，再繼續操作。注意：如果儲存格尚未執行，左側會顯示 `[]`。儲存格執行時，你會看到轉動的工作動畫。儲存格完成後，會顯示數字，例如 `[13]`。
9. 執行將 CSV 載入 DataFrame 的儲存格。等待檔案載入，並檢查前五列資料。這是我們要載入 BigQuery 的食譜資料，最終會用來做為食譜產生器的基礎。
10. 建立新的程式碼區塊，然後輸入下列註解。輸入註解後，移至下一行程式碼，系統應會顯示建議 `df.columns`。接受後執行儲存格。

```
# List the current DataFrame column names
```

我們剛才示範了在 Jupyter 筆記本中，如何透過兩種方式取得 Gemini Code Assist 的協助：程式碼儲存格上方的文字儲存格，或是程式碼儲存格內的註解。Jupyter 筆記本中的程式碼儲存格註解效果良好，但這個方法也適用於任何支援 Google Gemini Code Assist 的其他 IDE。

11. 我們來稍微清理一下資料欄。將資料欄 `Unnamed: 0` 重新命名為 `id`，並將 `link` 重新命名為 `uri`。使用您選擇的提示 > 程式碼技術建立程式碼，然後在滿意時執行儲存格。

```
# Rename the column 'Unnamed: 0' to 'id' and 'link' to 'uri'  
df.rename(columns={'Unnamed: 0': 'id', 'link': 'uri'}, inplace=True)
```

12. 移除 `source` 和 `NER` 欄，然後使用 `head()` 查看前幾列。再次請 Gemini 幫忙。執行最後兩行程式碼，並檢查結果。

```
# Remove the source and NER columns  
df.drop(columns=['source', 'NER'], inplace=True)  
df.head()
```

13. 讓我們看看資料集中有多少筆記錄。同樣地，請先選擇提示技巧，看看 Gemini 是否能協助您生成程式碼。

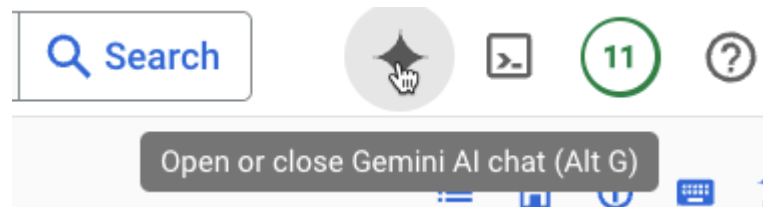
```
# Count the records in the DataFrame  
df.shape # count() will also work
```

14. 223 萬筆記錄可能比我們有時間做的食譜還多。在 Agent Builder 中建立索引的過程可能需要很長時間，因此不適合今天的練習。為求折衷，我們將取樣 15 萬份食譜，並以此為基礎進行作業。使用「提示」>「程式碼」方法，擷取範例

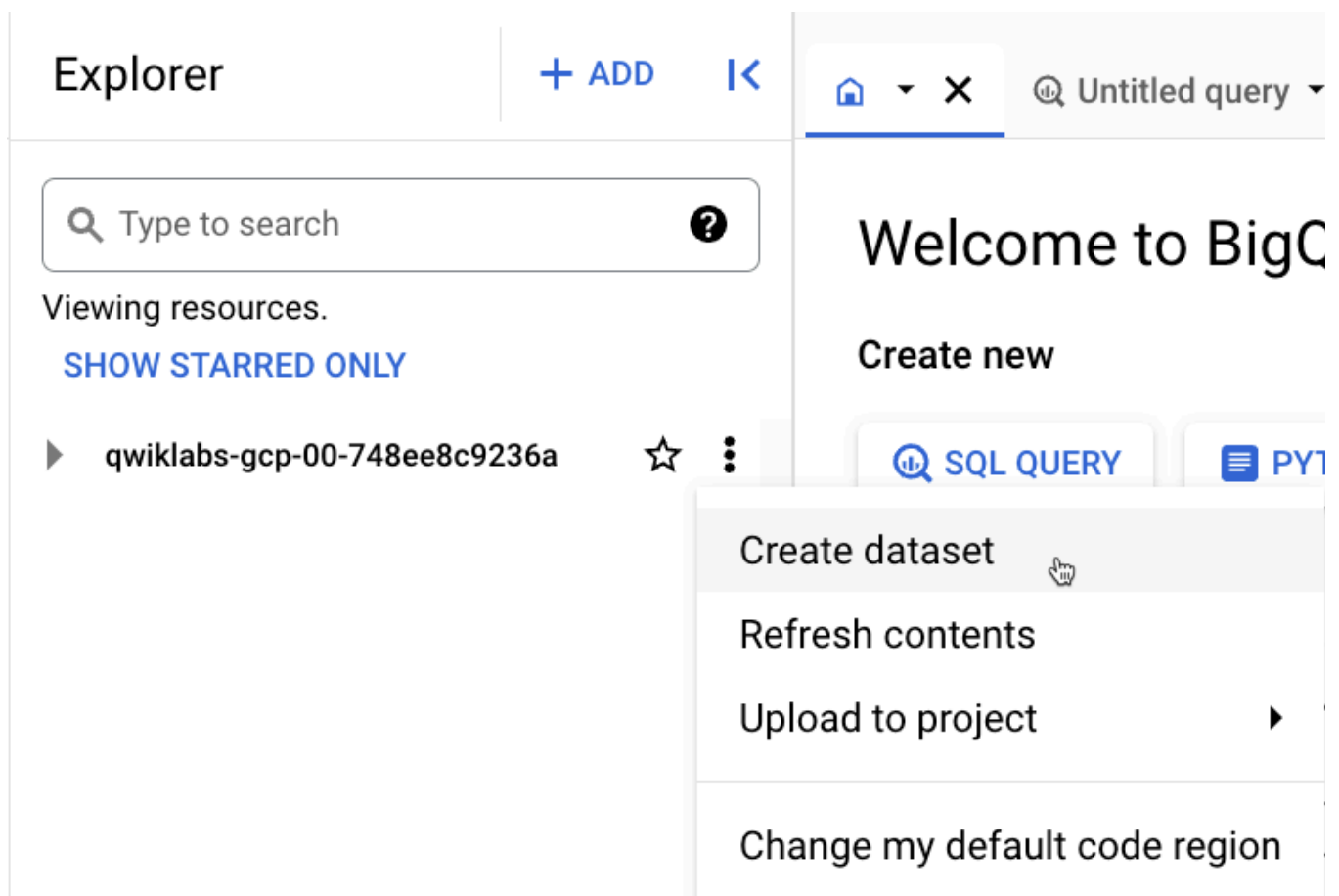
並儲存在名為 `dfs` (s 代表 small) 的新 DataFrame 中。

```
# Sample out 150,000 records into a DataFrame named dfs
dfs = df.sample(n=150000)
```

- 食譜來源資料已準備好載入 BigQuery。載入資料前，請先前往 BigQuery，準備好用來存放資料表的資料集。在 Google Cloud 控制台使用搜尋方塊，前往 **BigQuery**。您可以按一下滑鼠右鍵開啟 **BigQuery**，然後在新瀏覽器分頁中開啟。
- 如果 Gemini AI Chat 面板尚未顯示，請使用 Cloud Shell 右上角的 Gemini 標誌開啟。如果系統要求您再次啟用 API，請按下啟用鍵或重新整理頁面。執行提示：`What is a dataset used for in BigQuery?` 探索回應後，`How can I create a dataset named recipe_data using the Cloud Console?` 比較結果與下列幾個步驟。



- 在 BigQuery 的「Explorer」窗格中，點選專案 ID 旁邊的三點「查看動作」選單。然後選取「建立資料集」。



- 提供資料集和 `recipe_data` 的 ID。將位置類型保留為「美國」，然後「建立資料集」。如果收到資料集已存在的錯誤訊息，請直接繼續。在 BigQuery 中建立資料集後，請切換回筆記本並插入資料。
- 切換回 Colab Enterprise 中的資料整理筆記本。在新的程式碼儲存格中，建立名為 `project_id` 的變數，並用來保留目前的專案 ID。在操作說明的左上方，也就是「End Lab」按鈕下方，即可找到目前的專案 ID。您也可以在 Cloud 控制台首頁找到這項資訊。將值指派給 `project_id` 變數，然後執行儲存格。

```
# Create a variable to hold the current project_id  
project_id='YOUR_PROJECT_ID'
```

20. 使用提示 > 程式碼方法建立程式碼區塊，將 DataFrame `dfs` 插入剛才建立的資料集 `recipe_data` 中，並命名為 `recipes`。執行儲存格。

```
dfs.to_gbq(destination_table='recipe_data.recipes', project_id=project_id,  
if_exists='replace')
```

7. 在 Vertex AI Agent Builder 中建立食譜生成器的搜尋應用程式

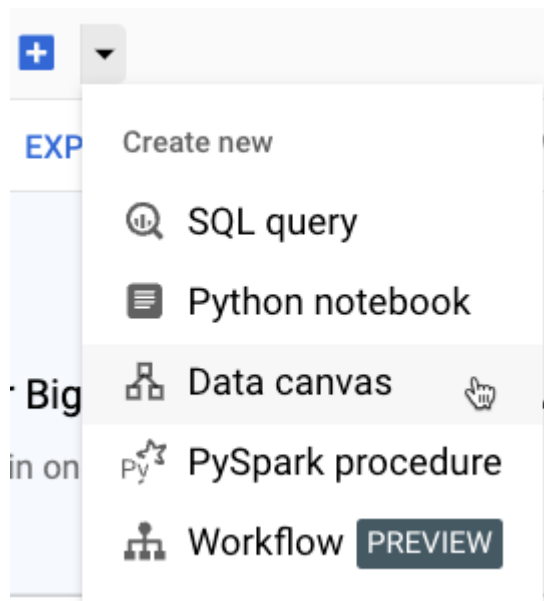
太棒了，我們已建立食譜資料表，現在要使用這個資料表，為食譜產生器建構基礎資料來源。我們將採用與烹飪聊天機器人類似的做法。我們將使用 Vertex AI Agent Builder 建立資料儲存庫，然後將該儲存庫做為搜尋應用程式的真實來源。

如要請 Gemini 在 Google Cloud 控制台中提醒您建立 Agent Builder 搜尋應用程式的步驟，或按照下列步驟操作，請隨意。

1. 使用搜尋功能前往 **Agent Builder**。開啟「資料儲存庫」並「建立資料儲存庫」。這次請**選取** BigQuery 資料儲存庫類型。
2. 在表格選取儲存格中，按下「瀏覽」並搜尋 `recipes`。選取表格旁的圓形按鈕。如果看到其他 `qwiklabs-gcp-...` 專案的食譜，請務必**選取**屬於您的食譜。注意：如果點選 `recipes` 而不是旁邊的單選按鈕，瀏覽器會開啟新分頁，並將您帶往 BigQuery 的資料表總覽頁面。只要關閉瀏覽器分頁，然後在 Agent Builder 中選取圓形按鈕即可。
3. 檢查其餘預設選項，但不要變更，然後點選「繼續」。
4. 在結構定義審查頁面中，檢查初始預設設定，但請勿變更任何項目。**繼續**
5. 將資料儲存庫命名為 `recipe-data`。編輯資料儲存庫 ID，並設為 `recipe-data-id`。**建立**資料儲存庫。
6. 使用左側導覽選單前往「應用程式」，然後「建立新應用程式」。
7. 按一下「網站搜尋功能」資訊卡上的「建立」。將應用程式命名為 `recipe-search`，然後點按「編輯」，將 ID 設為 `recipe-search-id`。將公司名稱設為 `Google`，然後按一下「繼續」。
8. 這次請檢查 **recipe-data** 資料來源。**建立**應用程式。

資料庫表格需要一段時間才能建立索引。在等待期間，我們來試用 BigQuery 的新功能「資料畫布」，看看能否找到一兩道有趣的食譜。

9. 使用搜尋框前往 **BigQuery**。在 BigQuery Studio 頂端，按一下最右側分頁標籤旁的向下箭頭，然後選取「資料畫布」。將區域設為 `us-central1`。



10. 按一下「搜尋資料」。在「資料畫布」搜尋框中搜尋 `recipes`，按下 `Enter/Return` 搜尋，然後按一下資料表名稱旁的「新增至畫布」按鈕。
11. 食譜資料表的視覺化呈現方式會載入 BigQuery 資料畫布。您可以探索資料表的結構定義、預覽資料表中的資料，以及檢查其他詳細資料。在表格表示法下方，按一下「查詢」。
12. 畫布會載入或多或少典型的 BigQuery 查詢對話方塊，但多了一個項目：查詢視窗上方有一個文字方塊，您可以在其中提示 Gemini 提供協助。讓我們看看能否在範例中找到一些蛋糕食譜。執行下列提示 (輸入文字並按下

Enter/Return 即可觸發 SQL 生成)：

Please select the title and ingredients for all the recipes with a title that contains the word cake.

13. 查看生成的 SQL 查詢。確認無誤後，**執行查詢**。

14. 還不錯吧！您可以先試試其他提示和查詢，再繼續下一個步驟。進行實驗時，請嘗試使用較籠統的提示，瞭解哪些提示有效，哪些無效。舉例來說，以下提示：

Do I have any chili recipes?

(別忘了執行新查詢) 傳回辣椒食譜清單，但未列出食材，直到我修改為：

Do I have any chili recipes? Please include their title and ingredients.

(我會在提示時說「請」，(我媽媽一定會很驕傲)。我發現其中一個辣椒食譜含有蘑菇，誰會在辣椒中加入蘑菇？我請 Gemini 協助排除這些食譜。

Do I have any chili recipes? Please include their title and ingredients, and ignore any recipes with mushrooms as an ingredient.

步驟 8 · 約 0 分鐘

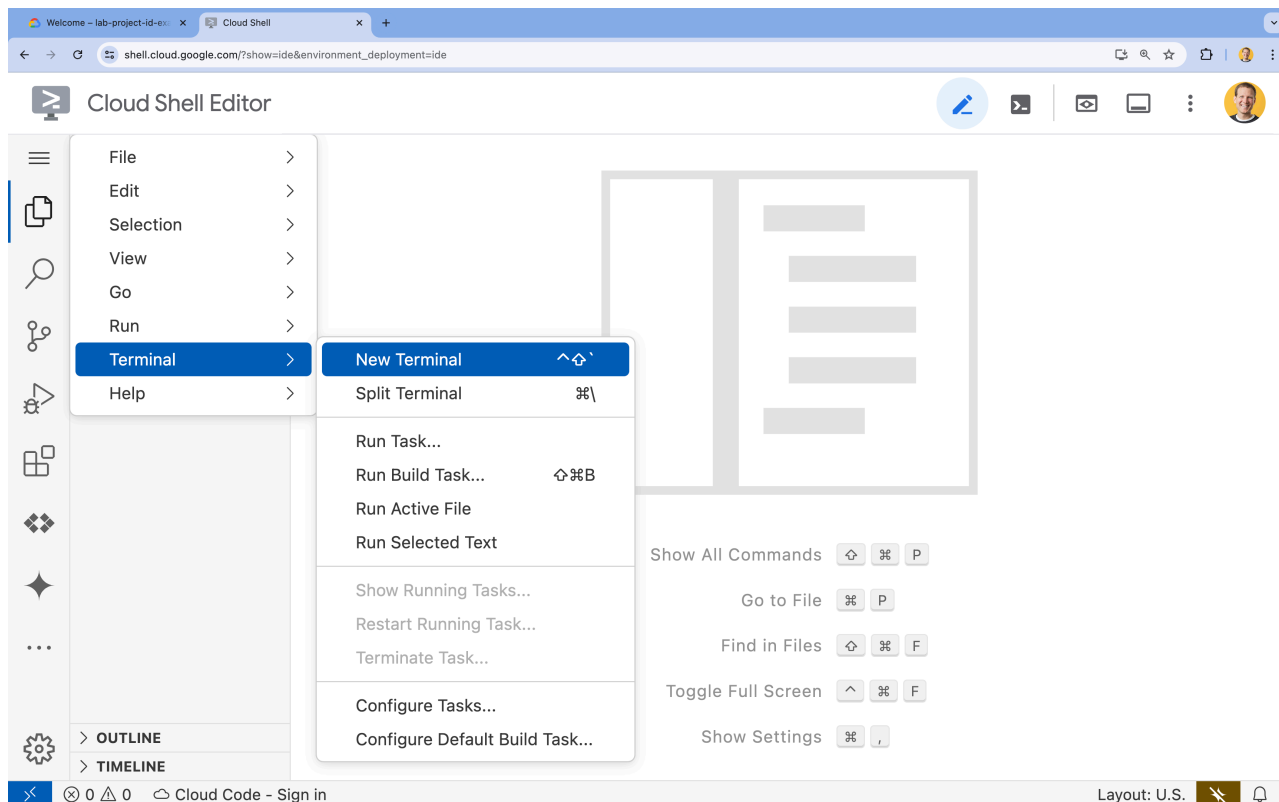
8. 開啟 Cloud Shell 編輯器

1. 前往 [Cloud Shell 編輯器](#)
2. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下漢堡選單



- 按一下「終端機」。
- 按一下「New Terminal」(新增終端機)



3. 在終端機中，使用下列指令設定專案：

- 格式：

```
gcloud config set project [PROJECT_ID]
```

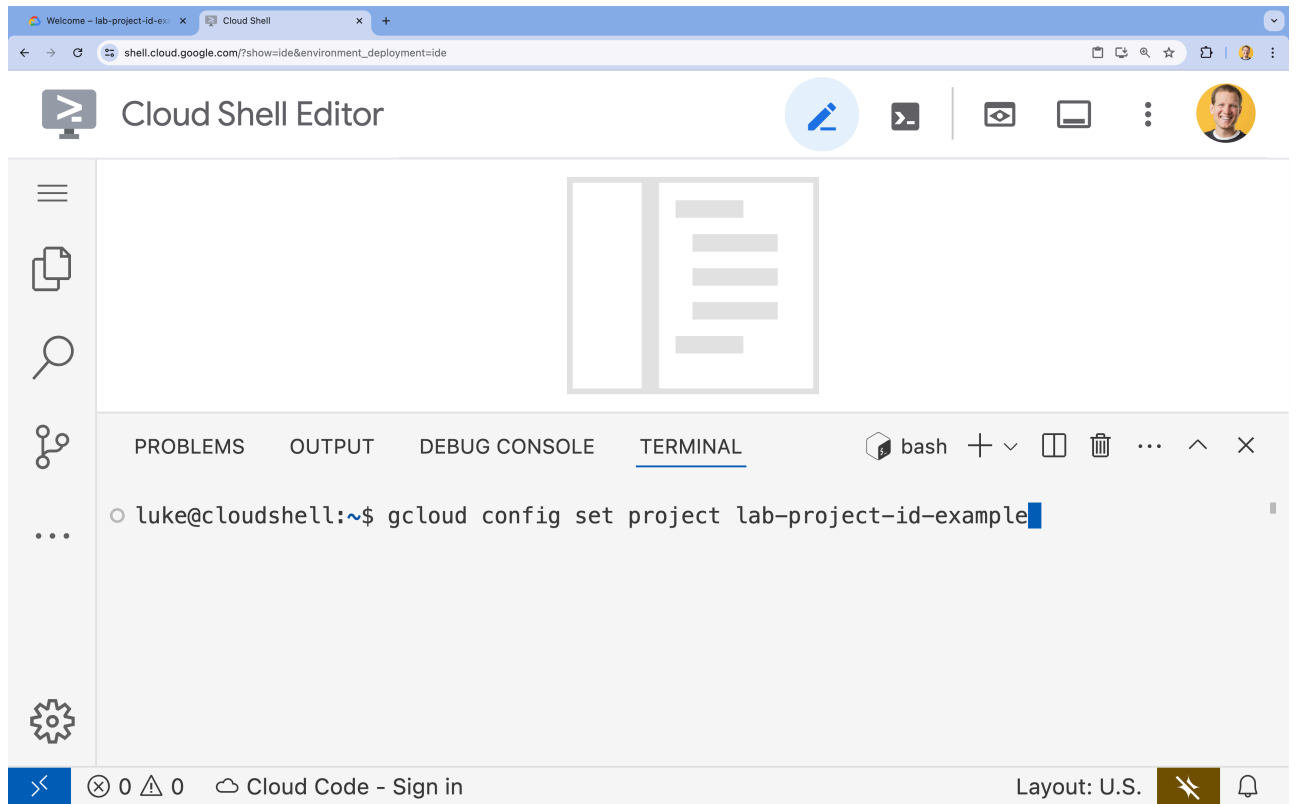
- 範例：

```
gcloud config set project lab-project-id-example
```

- 如果忘記專案 ID，請按照下列步驟操作：

- 您可以使用下列指令列出所有專案 ID：

```
gcloud projects list | awk '/PROJECT_ID/{print $2}'
```



4. 如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 `WARNING` 並收到 `Do you want to continue (Y/N)?` 提示，可能是輸入的專案 ID 有誤。按下 `N` 和 `Enter`，然後再次嘗試執行 `gcloud config set project` 指令。

步驟 9 · 約 0 分鐘

9. 啟用 API

在終端機中啟用 API：

```
gcloud services enable \  
  compute.googleapis.com \  
  sqladmin.googleapis.com \  
  run.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  networkconnectivity.googleapis.com \  
  servicenetworking.googleapis.com \  
  cloudaicompanion.googleapis.com
```

如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

這個指令可能需要幾分鐘才能完成，但最終應該會產生類似以下的成功訊息：

```
Operation "operations/acf.p2-73d90d00-47ee-447a-b600" finished successfully.
```

10. 在 Gemini 的協助下，規劃 Python 和 Streamlit 網頁應用程式的核心架構

兩個 Vertex AI Agent Builder 資料儲存庫都已建立索引，搜尋應用程式也差不多準備就緒，接下來就來建構網頁應用程式。

我們將在工作時運用 Gemini Code Assist。如要進一步瞭解如何在 Visual Studio Code 中使用 Gemini Code Assist，請參閱[這份文件](#)

1. 在 Cloud Shell 編輯器終端機中，執行下列指令來複製食譜應用程式存放區。

```
git clone https://github.com/haggman/recipe-app
```

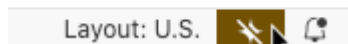
2. 執行這項指令，在 Cloud Shell 編輯器中開啟應用程式資料夾。

```
cloudshell open-workspace recipe-app/
```

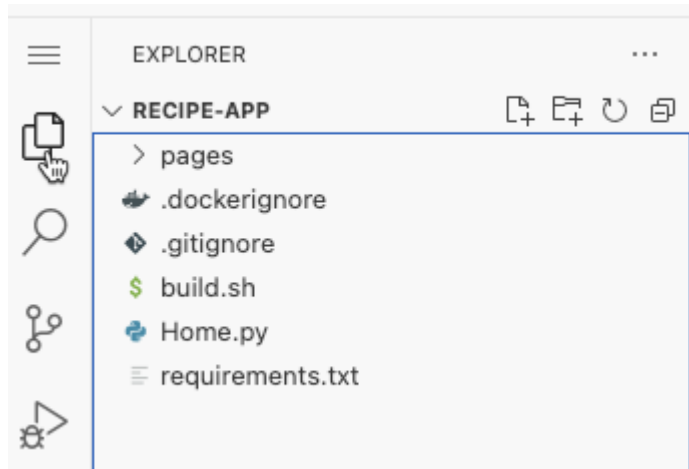
3. 在瀏覽複製的資料夾並開始處理網頁應用程式之前，我們需要讓編輯器的 Cloud Code 外掛程式登入 Google Cloud，並啟用 Gemini。現在就來瞭解吧！在編輯器的左下方，按一下「Cloud Code - Sign in」。如果沒有看到連結，請稍候再檢查一次。



4. 終端機視窗會顯示一長串網址。在瀏覽器中開啟網址，然後按照步驟授予 Cloud Code 存取 Google Cloud 環境的權限。在最後一個對話方塊中複製驗證碼，然後貼回 Cloud Shell 編輯器瀏覽器分頁中等待的終端機視窗。
5. 過一會兒，編輯器左下方的 Cloud Code 連結會變更為「Cloud Code - No Project」。按一下新連結即可選取專案。編輯器頂端應會開啟指令調色盤。點選「選取 Google Cloud 專案」，然後選取您的專案。稍待片刻，編輯器左下方的連結就會更新，顯示專案 ID。這表示 Cloud Code 已成功附加至工作專案。
6. Cloud Code 連線至專案後，即可啟用 Gemini Code Assist。在編輯器介面的右下角，按一下劃掉的 Gemini 標誌。編輯器左側會開啟 Gemini Chat 窗格。按一下「選取 Google Cloud 專案」。指令區塊面板開啟後，請選取專案。如果正確完成上述步驟 (且 Google 未進行任何變更)，現在應該會看到已啟用的 Gemini 對話視窗。



7. 太棒了！終端機、Gemini Chat 和 Cloud Code 設定都已完成，請開啟「Explorer」分頁，花幾分鐘探索目前專案中的檔案。



8. 在檔案總管中開啟 `requirements.txt` 檔案進行編輯。切換至 **Gemini** 對話窗格，然後提出以下問題：

From the dependencies specified in the requirements.txt file, what type of application are we building?

9. 因此，我們將使用 Python 和 Streamlit 建構互動式網路應用程式，與 Vertex AI 和 Discovery Engine 互動。目前先著重於網頁應用程式元件。如 Gemini 所述，[Streamlit](#) 是一種架構，可使用 Python 建構以資料為基礎的網頁應用程式。現在請詢問：

Does the current project's folder structure seem appropriate for a Streamlit app?

Gemini 在這方面往往會出錯。Gemini 可以存取您目前在編輯器中開啟的檔案，但無法查看整個專案。問題範例：

Given the below, does the current project's file and folder structure seem appropriate for a Streamlit app?

- build.sh
- Home.py
- requirements.txt
- pages
- Cooking_Advice.py
- Recipe_Search.py

取得更合適的答案？

10. 請提供更多有關 Streamlit 的資訊：

What can you tell me about Streamlit?

很好，我們可以看到 Gemini 提供了一份不錯的摘要，包括優點和缺點。

11. 如要瞭解缺點，可以問：

What are the major downsides or shortcomings?

請注意，我們不必說「of Streamlit」，因為 Gemini Chat 是對話式 (多輪)。由於我們正在進行即時通訊，Gemini 知道我們談論的內容。如要清除 Gemini 程式碼對話記錄，請使用 Gemini 程式碼對話視窗頂端的垃圾桶圖示。

11. 將網頁應用程式部署至 Cloud Run

太棒了，我們已建立核心應用程式結構，但這些結構是否都能正常運作？更重要的是，我們應該在 Google Cloud 中代管哪些內容？

1. 在 Gemini 對話視窗中提問：

```
If I containerize this application, what compute technologies in Google Cloud would be best for hosting it?
```

2. 請注意，如果您尚未在 IDE 中工作，也可以詢問 Google Cloud Assist。開啟 Google Cloud 控制台，然後開啟 Gemini Cloud Assist 並提出以下問題：

```
If I have a containerized web application, where would be the best place to run it in Google Cloud?
```

這兩組建議是否相同？你是否同意/不同意任何建議？請注意，Gemini 是生成式 AI 助理，就像真人助理一樣，有時提供的建議不一定符合需求。不過，在 Google Cloud 和程式碼編輯器中工作時，如果身邊有這麼一位幫手，效率會大幅提升。

3. 如果是無狀態的短期容器化網頁應用程式，Cloud Run 會是不錯的選擇。在程式碼編輯器的 Gemini 對話視窗中，嘗試輸入下列提示：

```
What steps would be required to run this application in Cloud Run?
```

4. 看來我們首先需要建立 Dockerfile。使用編輯器在專案資料夾的根目錄中建立名為 `Dockerfile` 的檔案。請務必不要誤放在 `pages` 資料夾中。開啟檔案進行編輯。
5. 我們將使用側邊 Gemini 對話面板建立 Dockerfile。請使用類似下方的提示。當結果顯示在對話視窗中時，請使用建議 Dockerfile 上方複製圖示旁的 +，將建議的程式碼插入 Dockerfile。

```
Create a Dockerfile for the application in the current folder. The dependencies are defined in requirements.txt and I want you to use the Python 3 slim bookworm base image.
```

Gemini 不一定會對相同提示詞傳回相同回覆。我第一次要求 Gemini 提供 Dockerfile 時，得到的檔案與我建議您使用的完全相同。我剛收到以下建議：

```
# Base image
FROM python:3-bookworm-slim

# Set working directory
WORKDIR /app

# Install dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    libpq-dev \
    gcc \
    python3-dev \
    && rm -rf /var/lib/apt/lists/*

# Install pip and virtualenv
RUN pip install --upgrade pip virtualenv

# Create virtual environment
RUN python3 -m venv venv

# Activate virtual environment
WORKDIR /app/venv/bin
RUN . activate

# Install Streamlit and libraries from requirements.txt
RUN pip install -r requirements.txt

# Copy application files
COPY . /app

# Expose port 8501 for Streamlit
EXPOSE 8501

# Start Streamlit app
CMD ["streamlit", "run", "main.py"]
```

這就是 Dockerfile。我會稍微簡化一下。我們不需要 apt-get 區段，因為 Python 需要的任何項目都已在基礎映像檔中。此外，在 Python 容器中使用虛擬環境會浪費空間，因此建議移除。公開指令並非必要，但可以執行。此外，它還嘗試啟動我沒有的 main.py。

6. 在 `recipe-app` 資料夾中建立名為 `Dockerfile` 的檔案，然後貼上以下內容：

```
FROM python:3.11-slim-bookworm

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["streamlit", "run", "Home.py"]
```

7. Gemini 可以透過對話視窗運作，也可以直接在程式碼檔案中使用註解，就像我們在資料整理筆記本中使用的註解一樣，而且也可以在 Windows 上使用 Control+i 鍵，或在 Mac 上使用 Command+i 鍵叫用。在 `Dockerfile` 中點選任意位置，然後使用適當的 Command+i / Control+i 指令啟動 Gemini。
8. 在提示中輸入下列內容。檢查並「接受」變更。

```
Please comment the current file.
```

這真是太棒了！您是否曾需要使用他人的程式碼，但必須先花時間瞭解對方沒有註解的程式碼，才能開始進行變更？讓 Gemini 幫你解決問題！

9. 接著詢問 Gemini，如何使用 Cloud Run 從目前資料夾中的 `Dockerfile` 建構及部署名為 `recipe-web-app` 的新映像檔。

```
How could I use gcloud to build a new Cloud Run service named recipe-web-app from the current directory?
```

10. 接著，請建構及部署應用程式。在終端機視窗中執行 `gcloud run deploy` 指令

```
gcloud run deploy recipe-web-app \
  --allow-unauthenticated \
  --source=. \
  --region=us-central1 \
  --port=8501
```

如果系統提示您建立 Artifact Registry 存放區，請按 `enter/return`

```
Deploying from source requires an Artifact Registry Docker repository to store built containers. A repository named cloud-run-source-deploy in region us-central1 will be created.
```

```
Do you want to continue (Y/n)?
```

11. 如果您觀看建構程序，會發現系統首先建構 Artifact Registry Docker 存放區。接著，Cloud Build 會使用本機資料夾中的 `Dockerfile` 建立容器映像檔。最後，Docker 映像檔會部署至新的 Cloud Run 服務。指令碼執行完畢後，您會取得可使用的 Cloud Run 測試網址。

在瀏覽器的新分頁中開啟傳回的連結。請花點時間瞭解應用程式的結構和頁面。很好，現在我們需要連結生成式 AI 功能。

12. 將「烹飪建議」頁面連結至我們的食譜搜尋 Agent Builder 應用程式

我們已準備好網頁應用程式的架構，但需要將兩個工作頁面連結至兩個 Vertex AI Agent Builder 搜尋應用程式。我們先從烹飪建議開始。

1. 請讓 Cloud Shell 編輯器分頁保持開啟。在 Google Cloud 控制台使用搜尋功能，前往 Vertex AI 中的「Chat」。
2. 在右側設定頁面窗格中，將模型設為 **gemini-1.5-flash-002**。將輸出權杖上限調到最高，模型就能視需要傳回較長的回覆。開啟「安全篩選設定」。將「仇恨言論」、「煽情露骨內容」和「騷擾內容」設為「封鎖部分」。將「危險內容」設為「封鎖少量」，然後按一下「儲存」。我們將「危險內容」的設定調低一些，因為 Gemini 可能會將刀具和切割相關內容誤解為暴力。
3. 滑動切換鈕啟用「基礎」，然後點選「自訂」。將基準來源設為 **Vertex AI Search**，並使用下列資料儲存庫路徑。將 YOUR_PROJECT_ID 變更為這些操作說明中「結束實驗室」按鈕附近的專案 ID，然後儲存基礎設定

```
projects/YOUR_PROJECT_ID/locations/global/collections/default_collection/dataStores/old-cookbooks-id
```

注意：如果發生錯誤，可能是因為您未將專案 ID 變更為實際專案 ID，或是您可能錯過了變更舊版食譜 Agent Builder 資料存放區 ID 的步驟。依序前往「Agent Builder」>「資料儲存庫」>「old-cookbooks」，查看實際的資料儲存庫 ID。

4. 測試幾則即時通訊訊息。或許可以從以下內容著手。你也可以試試其他幾種。

How can I tell if a tomato is ripe?

5. 模型運作正常，現在來試試程式碼。按一下「清除對話」，避免對話內容成為程式碼的一部分，然後按一下「取得程式碼」。

The screenshot shows the Vertex AI Chat interface. On the left, there's a chat window with a message: "A ripe tomato will have its skin loosen quickly, are smooth, uniformly ripe, and of moderate size. 1 spots. Uniformly ripe tomatoes are best for". Above the chat window are buttons for "Notes", "Clear Conversation", and "API reference". To the right of the chat window is an "Edit" button. On the right side of the interface is a settings panel. It includes a "Model" dropdown set to "gemini-1.5-flash-002", a "Region" dropdown set to "us-central1 (Iowa)", a "Temperature" slider set to 1, and an "Output token limit" slider set to 8192. At the bottom of the settings panel, there's a "Grounding" section with a toggle switch turned on, labeled "Source: Vertex AI Search", and a "Customize" button.

6. 在程式碼視窗頂端，按下「Open Notebook」(開啟筆記本)，即可在 Colab Enterprise 中實驗並完善程式碼，然後再整合至應用程式。

- 請花幾分鐘熟悉程式碼。讓我們進行幾項變更，以符合需求。開始之前，請先執行第一個程式碼儲存格，連線至運算資源並安裝 AI Platform SDK。區塊執行完畢後，系統會提示您重新啟動工作階段。請按照指示操作
- 前往從 Vertex AI Studio 擷取的程式碼。將方法 `multiturn_generate_content` 的名稱變更為 `start_chat_session`。
- 捲動至 `model = GenerativeModel(` 方法呼叫。現有程式碼定義了 `generation_config` 和 `safety_settings`，但實際上並未使用這些程式碼。修改 `GenerativeModel` 的建立方式，使其類似於：

```
model = GenerativeModel(  
    "gemini-1.5-flash-002",  
    tools=tools,  
    generation_config=generation_config,  
    safety_settings=safety_settings,  
)
```

- 最後，在 `chat = model.start_chat()` 下方的方法中新增最後一行，讓函式傳回 `chat` 物件。完成的函式應如下所示。注意：請勿將這段程式碼複製到筆記本中。這只是健全性檢查。

```
def start_chat_session():  
    vertexai.init(project="qwiklabs-gcp-02-9a7298ceaaec", location="us-central1")  
    tools = [  
        Tool.from_retrieval(  
            retrieval=grounding.Retrieval(  
                source=grounding.VertexAISearch(datastore="projects/qwiklabs-gcp-02-9a7298ceaaec/locations/global/collections/default_collection/dataStores/old-cookbooks-id"),  
            ),  
        ),  
    ]  
    model = GenerativeModel(  
        "gemini-1.5-flash-002",  
        tools=tools,  
        generation_config=generation_config,  
        safety_settings=safety_settings,  
    )  
    chat = model.start_chat()  
    return chat
```

- 捲動至程式碼儲存格底部，變更呼叫舊函式的最後一行，改為呼叫新函式名稱，並將傳回的物件儲存在 `chat` 變數中。對變更感到滿意後，請執行儲存格。

```
chat = start_chat_session()
```

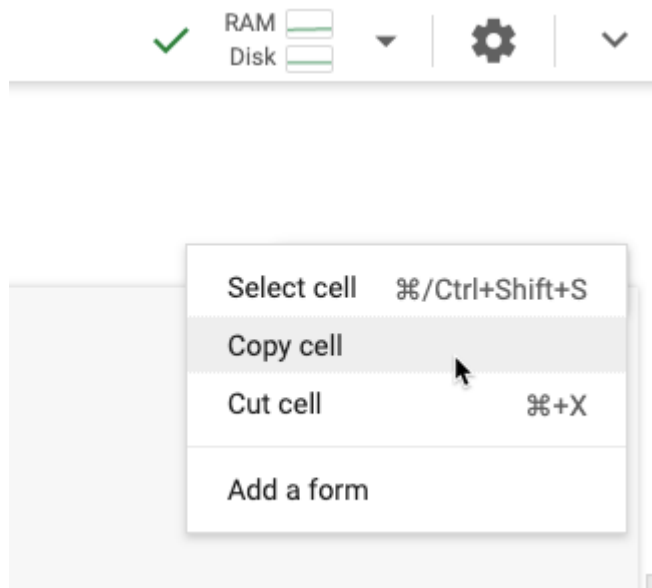
- 建立新的程式碼儲存格並加入註解 `# Use chat to invoke Gemini and print out the response`。移至下一行並輸入 `resp`，Gemini 應會自動完成程式碼區塊。將提示更新為 `How can I tell if a tomato is ripe?`。執行儲存格

```
response = chat.send_message("How can I tell if a tomato is ripe?")
print(response)
```

13. 這就是回應，但我們真正想要的是巢狀 `text` 欄位。修改程式碼區塊，只列印該區段，例如：

```
response = chat.send_message("How can I tell if a tomato is ripe?")
print(response.candidates[0].content.parts[0].text)
```

14. 很好，現在我們有了可用的即時通訊程式碼，接下來請將其整合至網頁應用程式。複製建立 `start_chat_session` 函式的程式碼儲存格的所有內容 (我們不需要測試儲存格)。按一下儲存格，然後點按右上角的三點選單，即可從該處複製



15. 切換至 Cloud Shell 編輯器分頁，然後開啟 `pages\Cooking_Advice.py` 進行編輯。

16. 找出留言：

```
#
# Add the code you copied from your notebook below this message
#
```

17. 將複製的程式碼貼到「Add the code」(新增程式碼) 註解下方。很好，現在我們有了透過 Gemini 的基礎呼叫來驅動聊天引擎的區段。現在將其整合至 Streamlit。

18. 找出註解正下方的註解程式碼區段：

```
#
# Here's the code to setup your session variables
# Uncomment this block when instructed
#
```

19. 取消註解這段程式碼 (直到下一個 `Setup done, let's build the page UI` 部分)，然後探索這段程式碼。這會建立或擷取即時通訊和記錄工作階段變數。

20. 接著，我們需要將記錄和對話功能整合至 UI。在程式碼中捲動，直到找到下列註解。

```
#  
# Here's the code to create the chat interface  
# Uncomment the below code when instructed  
#
```

21. 取消註解註解下方的其餘程式碼，並花點時間探索。如果需要，可以反白選取程式碼，請 Gemini 說明其功能。
22. 太棒了，現在請建構並部署應用程式。網址返回後，請啟動應用程式，然後試用烹飪顧問頁面。例如詢問如何挑選熟透的番茄，或是請機器人提供美味的甘藍菜料理方式。

```
gcloud run deploy recipe-web-app \  
  --allow-unauthenticated \  
  --source=. \  
  --region=us-central1 \  
  --port=8501
```

這真是太棒了！你的專屬 AI 烹飪顧問 :-)

14. (選用) 探索最終應用程式

請花幾分鐘的時間探索最終應用程式。

1. 在 Google Cloud 控制台中，使用搜尋功能前往 `Cloud Run`，然後點選「recipe-web-app」。
2. 找出應用程式測試網址 (位於頂端附近)，然後在新瀏覽器分頁中開啟。
3. 畫面上應會顯示應用程式首頁。請注意 Streamlit 提供的基本版面配置和導覽功能，其中 `pages` 資料夾中的 Python 檔案會顯示為導覽選項，而 `Home.py` 則會載入為首頁。前往「烹飪建議」頁面。
4. 不久後，系統就會顯示對話介面。再次提醒，Streamlit 提供的核心版面配置很不錯。
5. 嘗試問幾個與烹飪相關的問題，看看機器人如何運作。例如：

```
Do you have any advice for preparing broccoli?
```

```
How about a classic chicken soup recipe?
```

```
Tell me about meringue.
```

6. 現在來找一兩個食譜。前往食譜搜尋頁面，嘗試搜尋幾次。例如：

```
Chili con carne
```

```
Chili, corn, rice
```

```
Lemon Meringue Pie
```

```
A dessert containing strawberries
```

15. 恭喜！

您已建立應用程式，並運用 Vertex AI Agent Builder 應用程式。您已瞭解 Gemini Cloud Assist、Gemini Code Assist，以及 BigQuery 資料畫布的自然語言轉 SQL 功能。太棒了！

清除所用資源

Cloud SQL 沒有免費方案，如果您繼續使用，系統會向您收費。您可以刪除 Cloud 專案，以免產生額外費用。

不使用服務時，Cloud Run 不會收費，但您可能仍須支付容器映像檔在 Artifact Registry 的儲存費用。刪除 Cloud 專案後，系統就會停止對專案使用的所有資源收取費用。

如要刪除專案，請按照下列步驟操作：

```
gcloud projects delete $GOOGLE_CLOUD_PROJECT
```

您也可以從 Cloud Shell 磁碟刪除不必要的資源。您可以：

1. 刪除 Codelab 專案目錄：

```
rm -rf ~/task-app
```

2. 警告！這項操作無法復原，如要刪除 Cloud Shell 中的所有內容來釋出空間，可以[刪除整個主目錄](#)。請務必將要保留的內容另存到其他位置。

```
sudo rm -rf $HOME
```
